

Regression Tree Progetto MAP 2026

Progettazione, implementazione e verifica di un sistema Java per apprendere, visualizzare e usare alberi di regressione.

Java 8+

Swing

TCP/IP

JDBC · MySQL

JUnit 4

INDICE

1	Obiettivo e architettura	2
2	Algoritmo e modello software	3
3	Interfaccia grafica	4
4	Modalità client/server	5
5	Verifica e qualità	6
6	Esecuzione, limiti e conclusioni	7

Due modalità, ben separate

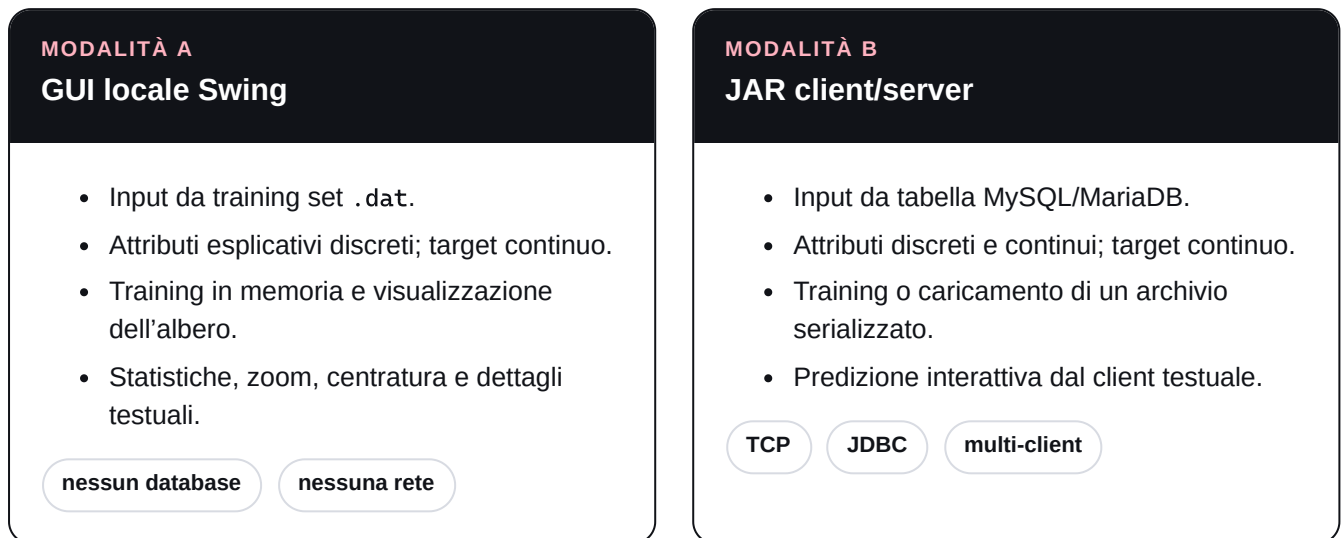
GUI locale su file .dat e sistema JAR
client/server su database.

Risultato verificato

81 test JUnit, 0 errori e 0 fallimenti nella verifica
locale.

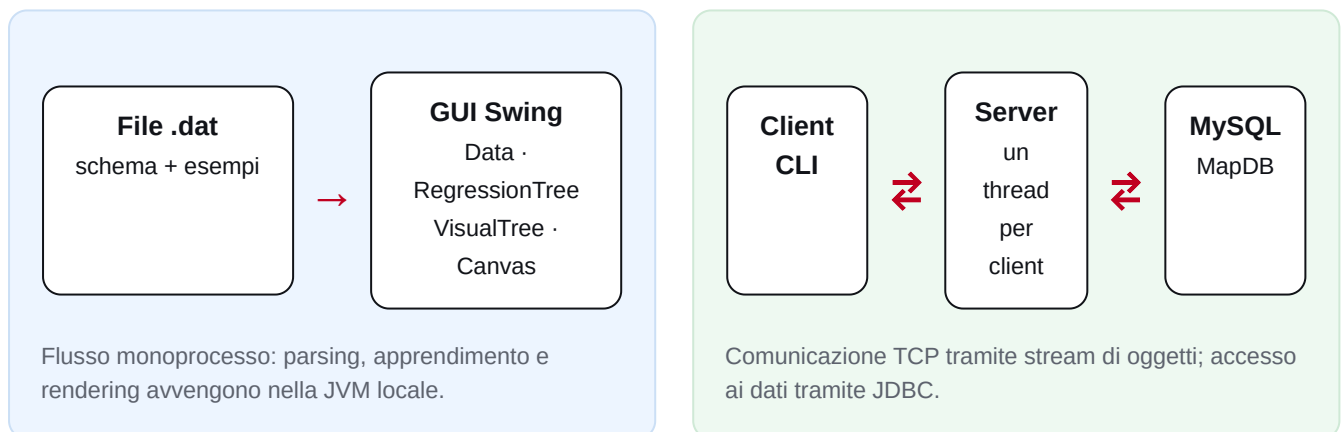
01 Obiettivo e architettura

Il progetto stima un **target numerico** tramite un albero di regressione. Il repository espone due modalità autonome: condividono i concetti di dominio, ma non comunicano tra loro.



Confine essenziale. La GUI non effettua predizioni e non interroga il database. I JAR non leggono file .dat e non mostrano un'interfaccia grafica.

Vista a componenti



Tecnologie e artefatti

AREA	TECNOLOGIA	ARTEFATTO PRINCIPALE
Dominio	Java SE	src/data, src/tree
Interfaccia	Swing/AWT, rendering custom	estensioni/*.java
Distribuzione	JAR, Socket, Object Streams	distribution/client, distribution/server
Persistenza dati	JDBC + Connector/J 8.0.17	map6_setup.sql
Verifica	JUnit 4.12	tests/

02 Algoritmo e modello software

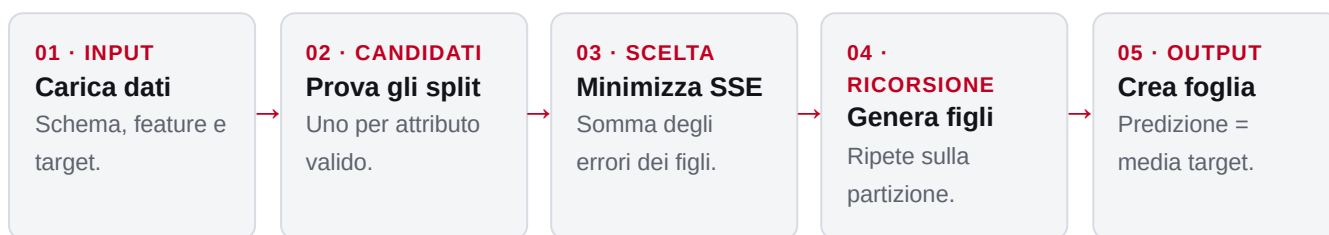
Criterio di apprendimento

Ogni nodo copre una partizione di esempi. L'errore memorizzato come **variance** è, in realtà, la **somma degli errori quadratici** (SSE) rispetto alla media del target.

$$\hat{y}_S = (1 / |S|) \cdot \sum_{i \in S} y_i$$

$$SSE(S) = \sum_{i \in S} (y_i - \hat{y}_S)^2$$

Per ogni attributo candidato si costruiscono le partizioni e si sceglie lo split che minimizza **Σ SSE(figlio)**. Una foglia restituisce la media $\hat{y}$ della propria partizione.



Arresto

La soglia è il **10% del training set**, con divisione intera. Il nodo diventa foglia quando la partizione non supera la soglia oppure quando non produce più di un ramo valido.

Differenza tra le modalità

Il codice locale valuta split **discreti**. Il server distribuito include anche `ContinuousNode` e scarta feature continue senza valori distinti.

Responsabilità delle classi

PACKAGE / CLASSE	RESPONSABILITÀ	COLLABORAZIONI
<code>data.Data</code>	Carica e valida il training set; espone feature e target; ordina partizioni.	<code>Attribute</code> , <code>DiscreteAttribute</code> , <code>ContinuousAttribute</code>
<code>tree.Node</code>	Definisce intervallo di esempi e SSE comune ai nodi.	<code>SplitNode</code> , <code>LeafNode</code>
<code>tree.SplitNode</code>	Descrive attributo, rami e costo dello split.	<code>SplitInfo</code> , nodi discreti/continui
<code>tree.RegressionTree</code>	Coordina selezione dello split, ricorsione, regole e predizione.	<code>Data</code> , gerarchia <code>Node</code>
<code>estensioni.VisualTree</code>	Adatta il modello ai soli dati richiesti dalla vista.	<code>RegressionTreeCanvas</code>
<code>server.ServerOneClient</code>	Gestisce il protocollo e lo stato della singola sessione.	<code>Socket</code> , <code>Data</code> , <code>RegressionTree</code>

Scelte progettuali

Ricorsione

Albero composto da sotto-alberi omogenei.

Polimorfismo

`Node` e `SplitNode` astraggono foglie e split.

Adapter visuale

La GUI non modifica le classi del dominio.

03 Interfaccia grafica

L'estensione Swing rende leggibile la struttura appresa senza introdurre dipendenze esterne. Il training parte da un file locale e il risultato viene disegnato su un canvas navigabile.

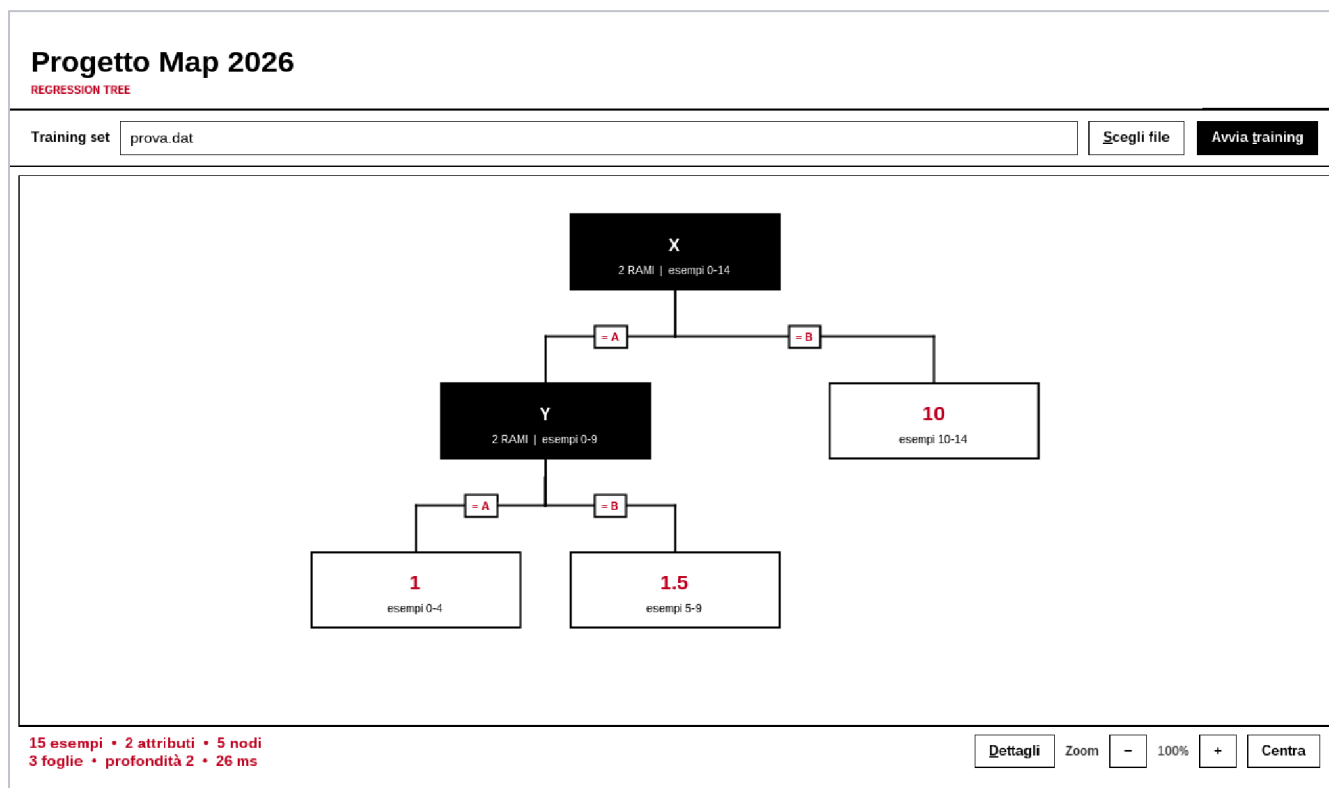


Figura 1. Training di prova.dat: nodi di split neri, condizioni sui rami e predizioni in rosso nelle foglie.

Flusso utente

1. Inserimento o selezione del file .dat.
2. Parsing e validazione tramite Data.
3. Costruzione di RegressionTree.
4. Conversione in VisualTree e rendering.

Rendering

- layout ricorsivo basato sulla larghezza dei sotto-alberi;
- zoom limitato al 50–150% e centratura della radice;
- scroll per alberi più grandi del viewport;
- conteggio di nodi, foglie, profondità e tempo.

Threading. Finestra e azioni sono create sull'Event Dispatch Thread di Swing.

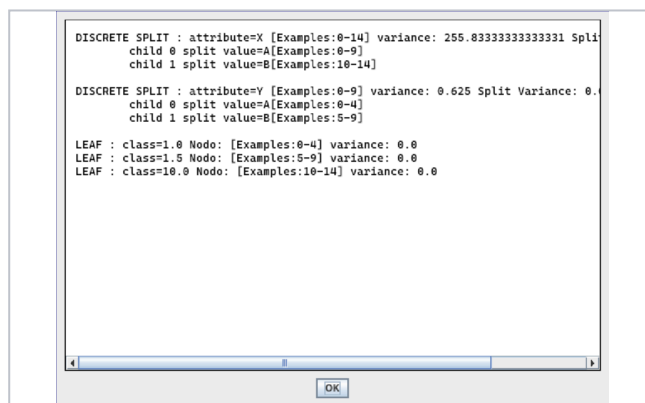
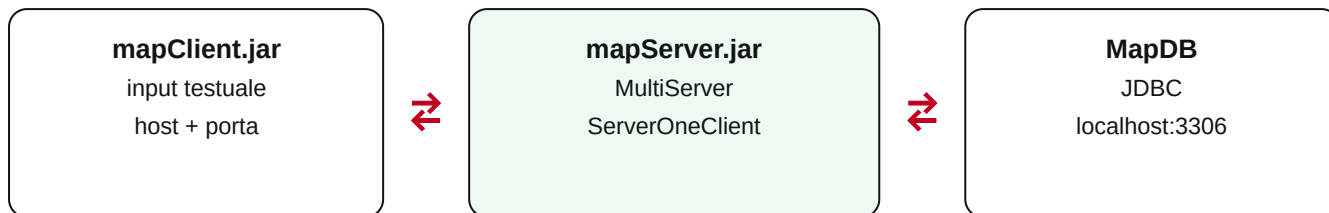


Figura 2. Vista testuale con partizioni, SSE e valore predetto.

Formato minimo

```
@schema 2  
@desc X A,B  
@desc Y A,B  
@target class  
@data 4  
A,A,1.0  
A,B,1.5  
B,A,10.0  
B,B,10.0
```

04 Modalità client/server



Concorrenza e sessione

MultiServer accetta connessioni in ciclo. Per ogni socket crea un `ServerOneClient`, sottoclasse di `Thread`, che conserva training set, albero e nome archivio della sessione.

Persistenza

Dopo il training il server serializza automaticamente l'albero in un file con il nome della tabella. Un'esecuzione successiva può caricarlo senza interrogare il database.

Protocollo applicativo

CODICE	OPERAZIONE	SCAMBIO PRINCIPALE
0	Carica dati	nome tabella → OK / messaggio di errore
1	Apprendi e salva	costruzione albero → OK / errore I/O
2	Carica archivio	nome file → OK / errore
3	Predici	QUERY + rami ≠ indice; infine OK + valore

```
$ java -jar distribution/client/mapClient.jar localhost 8080
```

```
Socket[addr=localhost/127.0.0.1,port=8080,localport=60168]
Learn Regression Tree from data [1]
Load Regression Tree from archive [2]
1
File name:
provaC
Starting data acquisition phase!
Starting learning phase!
Starting prediction phase!
0:X=A
1:X=B

0
0:Y≤2.0
1:Y>2.0

0
Predicted class:1.0
Would you repeat ? (y/n)
n

Esecuzione terminata.
```

Figura 3. Sessione reale: training dalla tabella `provaC`, scelta dei rami e predizione finale pari a 1.0.

Database

MapDB, utente MapUser, porta 3306.

Dataset inclusi

`provaC` (15 righe) e `servo` (167 righe).

Porta MAP

8080 predefinita, configurabile all'avvio.

05 Verifica e qualità

La suite è stata ricompilata ed eseguita sul contenuto corrente del repository. Il risultato è ripetibile con JUnit 4 e non richiede il database per i test locali.

81

test eseguiti

0

fallimenti

0

errori

13

classi di test

Distribuzione dei casi



AREA	CASI	COMPORTAMENTI VERIFICATI
data	17	schema, parsing, accesso ai valori, ordinamento, input errati
tree	26	SSE, split discreti, foglie, ricorsione, regole, predizione ed errori
utility	15	lettura di tipi, limiti numerici, EOF e conteggio errori
MainTest	4	flussi console validi e gestione eccezioni
estensioni	19	adapter visuale, layout canvas, zoom, training e stati GUI

Evidenza di esecuzione

```
$ javac -cp junit-4.12.jar:hamcrest-core-1.3.jar ...
$ java -cp build/report-tests:... org.junit.runner.JUnitCore [13 classi]
JUnit version 4.12
.....
Time: 0.953

OK (81 tests)
```

✓ Punti coperti bene

- casi nominali e valori limite;
- file mancanti e formati non validi;
- split senza ramificazione utile;
- rendering e interazioni Swing sull'EDT.

⚠ Perimetro non coperto

- integrazione JDBC con un DB reale;
- protocollo socket end-to-end e concorrenza;
- compatibilità degli archivi tra versioni;
- misure automatiche di code coverage.

Ambiente di verifica: OpenJDK 26.0.2 su Linux, display grafico disponibile. I JAR distribuiti hanno bytecode Java 8 (major version 52).

06 Esecuzione, limiti e conclusioni

Avvio rapido

GUI LOCALE

```
mkdir -p build/gui
javac -d build/gui \
  src/data/*.java src/tree/*.java \
  src/utility/*.java estensioni/*.java
java -cp build/gui
  estensioni.RegressionTreeGUI
```

Avviare dalla radice per usare direttamente prova.dat.

CLIENT/SERVER

```
mysql -u root -p \
  <
  distribution/server/sql_file/map6_setup.sql
cd distribution/server
java -jar mapServer.jar 8080
# secondo terminale, dalla radice:
java -jar distribution/client/mapClient.jar \
  localhost 8080
```

Il Connector/J deve restare accanto a mapServer.jar.

Limiti tecnici e mitigazioni

Separazione funzionale

GUI solo locale; predizione solo client/server. Mitigazione: presentare le due modalità come strumenti distinti.

Sicurezza di rete

Protocollo senza TLS o autenticazione. Mitigazione: uso esclusivo su rete locale o fidata.

Configurazione fissa

Database e credenziali sono incorporati. Mitigazione: centralizzare la configurazione in variabili d'ambiente o file esterno.

Serializzazione Java

Archivi sensibili alla versione e non adatti a input non fidati. Mitigazione: validare i percorsi e usare un formato esplicito.

Training sincrono nella GUI

Dataset grandi possono bloccare l'EDT. Mitigazione: spostare il training in uno `SwingWorker`.

Riproducibilità server

Nel repository analizzato il client/server è disponibile solo come JAR; il relativo sorgente non è presente. Mitigazione: versionare anche i sorgenti della build distribuita.

Valutazione finale

Esito. L'implementazione separa correttamente dominio, visualizzazione e distribuzione. L'algoritmo è verificabile, la GUI rende l'albero comprensibile e il server supporta più client, persistenza e predizione. Le priorità successive sono test end-to-end, configurazione esterna e training asincrono.

Tracciabilità documentale

Manuale operativo	docs/manuale_utente.pdf
UML GUI / locale	docs/diagramma-classi.svg · docs/diagramma-classi.pdf
UML client/server	docs/diagramma-classi-client-server.svg · docs/diagramma-classi-client-server.pdf
Dataset di esempio	prova.dat · servo.dat · distribution/server/sql_file/map6_setup.sql
Baseline analizzata	david @ a016b53